演讲稿

Page 1 | Opening

大家好，随着 AI 这两年的快速发展，我们使用 AI 的方式其实也在不断变化，从最开始比较简单的问答，到现在越来越多地尝试把 AI 放进真实的工作任务和流程里面。和大家分享一下我们在这个使用过程中探索的一些 steps：我们怎么从最开始和 AI 来回聊天，逐渐走到一个可以重复执行、检查和改进的工作方式，以及在这个过程中怎么节省token

Page 2 | From Chat to Reliable Work

如果把所有的内容放在一条线上，大概是这样的。一开始，我们有一个问题，就直接去问 AI。但当任务开始变得复杂之后，我们会发现只靠 Chat 往往是不够的。第一步，我们需要给 AI 一个更清楚的 Structured Prompt，让 AI 知道到底要做什么、按照什么规则做，以及最后要交付什么。但 Prompt 写清楚以后，问题还没有结束。随着文件、修改和对话越来越多，我们会遇到第二个问题：AI 现在到底在用哪些信息？哪些要求还是有效的？ 这就进入了 Managed Context。接下来还有第三个问题：即使 AI 用对了信息，它给我的结果到底能不能直接用、能不能验证？所以我们还需要考虑 Verifiable Output。

当 Prompt、Context 和验证方式都逐渐稳定以后，我们就可以把一次成功的做法保存成可重复使用的模版。如果任务需要经常重复，就可以进一步变成 Workflow。而如果下一步不是提前固定好的，而是需要 AI 根据当前结果自己决定接下来做什么，这时候才开始进入 Agent。

所以这条线上其实有三个问题：

- 我们怎么给 AI 更清晰、更有效的指令?
- 在 Context 变得混乱之前，我们应该怎么管理它?
- 我们怎么把有效的 AI 交互，变成可以重复执行的 Workflow，甚至进一步变成 Agent?

今天这一部分主要涉及在前面这段Prompt、Context 和 Output 这部分。

页面下面还有一个颜色说明：白色是讲解用的 case dialogue 或输出示例；灰色是已经填入具体案例内容的 structured prompt；浅蓝色是可以调整后复用的 prompt。后面看到浅蓝色，就可以把它当成一个实际操作的入口，再根据自己的任务修改。

Page 3 | From Chat to Structured Prompt

我们先把这个 case 过一下。这里用的是一组 mock data，假设明天要做 deliverable review，Leader 发来五个文件，希望我们提前 review 一下 AE summary，看看有没有 issues or inconsistencies。

这五个文件各自是干什么的？ADSL 是 subject-level data，我们用它确认哪些 subject 属于 safety population，以及每个 treatment group 的 denominator。ADAE 是 AE data，我们从这里检查每个 subject 的 AE，以及对应的 SOC 和 PT。RTF 是这次要 review 的 table，analysis specification 告诉我们应该按照什么规则统计，program log 则帮助我们检查运行时的 warning。

所以这个任务其实是拿着 source data 和 specification，去核对已经生成的 AE table。比如 population 对不对？counts 和 percentages 能不能对上？排序有没有按照 spec？log 里面的 warning 有没有影响结果？

但我们让 AI 做的时候，很可能只把文件丢进去，然后说一句："Please review these files and tell me if there are any issues."大家看这里 AI 的回答，它确实提到了 Placebo issue，也提到了 counts、percentages 和 program log。乍一看，好像该说的都说了。

我们先看绿色部分。它说"appears to have a Placebo issue"，那到底只是怀疑，还是已经能够确认？它又说"some percentages and a few counts"，那具体是哪几行，现在是多少，应该是多少？最后说"review with the programmer"，那 programmer 接下来到底要检查什么，修正以后又要怎么确认？这些还没有讲清楚。

再看黄色部分。我们自己的 request 其实也没有定义清楚："any issues"要检查到什么程度？"tell me"是要一段总结，还是一个可以逐项处理的 issue list？如果这件事由我们自己来做，很多要求可能已经在脑子里了。但 AI 只看到了这句话，它不知道我们默认的 review standard 是什么。

所以接下来，我们就用同一个 case 看一下，这些 hidden assumptions 怎么写进 prompt。

Page 4 | From Vague Request to Structured Prompt

> **讲图提示：**先出现左侧 CTIRO，再出现中间说明和短例子，然后出现右侧完整 prompt；继续点击依次高亮 C、T、I、R、O 的对应段落，最后出现 AI 的省略号。右侧 prompt 出现时 C 已高亮，也可点击左侧条目切换。

这页左边把刚才省略的东西拆成了五部分：Context、Task、Input、Rules 和 Output。我们先看左边这五项，再看中间这个简单例子，最后把它们放回右边完整的 prompt 里面。

Context 是为什么要做。这里就是为了明天的 deliverable review。Task 是具体做什么：核对 AE summary，找出能够确认的问题和不一致。Input 则要把刚才那五个文件的角色说清楚，不能默认 AI 知道哪个是 source，哪个是 output，哪个决定统计规则。

中间这句话想表达的就是，我们平时默认的一些工作规则，需要主动告诉 AI。比如下面这句“Check whether the AE summary is correct.”看起来很明确，但 correct 到底怎么判断？这就要落到 Rules 上。

现在看右边这段灰色的 prompt，它是刚才这个 AE case 的完整写法。左边每一项都能在右边找到对应的位置。讲到 Context，就看它有没有说明 review 的目的；讲到 Input，就看五个文件有没有分清角色。到了 Rules，我们再把这个 case 具体过一下。

我们先过一下 denominator。按照这个 case 的 spec，safety population 要用 SAFFL=Y。ADSL 里面 Placebo 一共有六个 subject，但 SUBJ012 的 SAFFL=N，所以 safety denominator 应该是五，而 RTF 表头写的是六。这里我们就可以给出一个具体 finding：Placebo N=6，应该改成 N=5，依据是 ADSL 和 specification。

再看 Any AE。我们要数的是发生 AE 的 subject，不是 ADAE 的记录条数。同一个 subject 有多条 AE，在 Any AE 这一层也只能算一次；在每个 SOC、每个 PT 里面，同样要分别去重。按这个规则，Placebo safety population 里面有四个人发生过 AE，所以应该是 4/5，也就是 80.0%。但现在 table 写的是 5 (83.3%)。这说明这里不仅 denominator 不一致，numerator 也需要核对，不能只把所有百分比的分母换一下就算完成。

同样，Placebo 的 Gastrointestinal disorders 和 Nausea，在 safety population 里各涉及一个 subject，应该是 1 (20.0%)，而 table 都写成了 2 (33.3%)。另外，spec 还规定 SOC 按字母排序，SOC 里面的 PT 按跨 treatment group 的 subject 总数降序排列，同数再按字母排。因此 Nervous system disorders 下面，Headache 应该放在 Dizziness 前面。

这些就是我们需要写给 AI 的 Rules：选哪个 population、在哪个层级数 subject、用什么 denominator、保留几位小数，以及按照什么规则排序。log 也要看。这里 log 已经提示 Placebo denominator 是在 SAFFL filter 之前生成的，但具体代码怎么改、其他 count 的差异怎么造成，仍然需要 programmer 检查源码，并通过 rerun 确认，不能直接把所有问题都归到一个原因上。

最后是 Output。我们希望拿到的就不只是“可能有问题”，而是一张 review table：finding 是什么，severity 怎么判断，evidence 在哪里，current value 和 corrected value 分别是多少，影响哪些 rows，以及 next step 是什么。比如刚才 Any AE 那一行，就可以写清楚：当前 5 (83.3%)，独立重算为 4 (80.0%)，依据是 ADSL、ADAE 和 spec；下一步请 programmer 核对 population 和 counting logic，修正后重新生成 table，再做 QC。

所以我们看右边这个完整 prompt，其实任务没有变，还是 review 同一张 AE table。变化在于，我们把刚才实际用到的文件角色、统计规则和交付要求写出来了。这样 AI 才有一个明确的标准去执行，我们也有一个标准去检查它的结果。

Page 5 | Align Before Execute

但真正接到任务的时候，我们可能还没有把刚才这些东西全部想清楚。所以这页讲的是，如果手里只有一个 rough idea，怎么一步一步把它变成刚才那个 prompt。

先看左边这些 common problems。还是这个 AE case，如果 AI 只说“有一些 Placebo issues”，那结果太 generic，我们就要求它列出具体 row、current value 和 corrected value。如果拿到的是一大段文字，格式不方便 review，就给它 Finding、Evidence、Correction、Next Step 这样的 output schema。如果是给 Leader 看，我们可以要求先讲最重要的 finding，计算细节放到后面。每次都针对一个具体问题修改，而不是只说“再好一点”。

中间是更早一步：连我们自己都还不确定缺什么的时候，可以先让 AI 帮忙整理需求。比如我只有那五个文件和 Leader 的一句话，就先告诉它：“Please do not execute this task directly.”先检查我的任务有没有缺 background、input、rules、output format 或 validation criteria，再问最多三个最重要的问题。

放到这个 case 里面，值得先澄清的可能就是：这次需要独立重算所有 n (%)，还是只做表面一致性检查？统计规则以哪个版本的 spec 为准？最后需要一段 summary，还是逐项可处理的 QC issue list？这些是我们可以要求 AI 检查的问题，不是说 AI 已经在这次对话里实际问过了。文件里已经明确的规则，也应该先读取，不需要再让人重复提供。

右边是当 prompt 基本明确以后，执行前再对齐一次。我们可以让 AI 先说它准备怎么做：先读 spec；再用 ADSL 确认 SAFFL=Y 的 safety denominator；用 ADAE 按 subject 重算 Any AE、SOC 和 PT；然后和 RTF 逐行对照，检查百分比、排序及 log warning，最后生成 issue list。

如果它在这一步说“我准备直接统计 ADAE 的记录数”，我们就能在执行之前发现它把 event count 和 subject count 混淆了。现在纠正，只需要明确一句“按 distinct subject 统计”；等整张表都算完再改，就要重新走一遍。

所以我们不需要一开始就写出一个完美的 prompt。上面这五步 Define → Generate → Review → Refine → Reuse，就是先定义任务，形成第一版，再拿具体结果来检查和修正。等到这个 AE review 的输入、规则和输出结构稳定了，再把做得好的部分保存成 template，下次换一组文件继续用。

到这里，我们先解决了怎么让 AI 知道要做什么。但当后面开始反复上传文件、修改要求、追问结果时，它现在到底在参考哪一份信息？这就是下一段要讲的 Context。

Page 6 | Context — AI Responds to More Than Your Last Message

> **讲图提示：**先展示用户短消息和 Context Container，再出现下方说明、右侧 Common Problems，最后出现总结。

前面我们已经把 task 说清楚了，但在实际工作里，我们还会继续上传文件、修改要求、追问结果。所以接下来还有一个问题：为什么最开始说清楚的事情，聊到后面又可能变乱？这里就要看 Context。

比如我们现在只说一句："Make the AE QC brief shorter."从我们输入的内容看，这句话很短。但 AI 理解它的时候，可能还会参考最开始的 AE review 任务、上传的五个文件、前面生成的 findings、我们后来修改的要求，以及工具的输出。

所以大家看中间这个 container，里面不只有最后一句话。Original Task、Conversation History、Uploaded Files、Previous Outputs、User Corrections，这些都可能影响接下来怎么回答。不同产品实际保留、检索、总结或者截断历史信息的方式也会不同，不是说每次都一定会把全部聊天原样放进去。

这也意味着，一个短的 input，不一定对应一个很小的 context。输入消息长度和系统实际使用的上下文是两回事，不能只看我们打了几个字，就判断这次调用的 token 用量或产品的计费。

右边是可能出现的几种表现。比如我们本来要求先看 counts 和 percentages，后面却主要在讨论 formatting；或者已经换成当前的 AE table，它又用了之前的版本；又或者前面说好 one-page，后来这个限制不见了。

所以问题有时候不是信息不够，而是信息太多以后，当前真正重要的内容没有被突出出来。下一页我们就沿着同一个 AE review，看这些信息是怎么一步一步混在一起的，以及发现以后具体可以做什么。

Page 7 | How Context Gets Polluted

> **讲图提示：**第一次点击显示左侧全部对话。之后依次出现 Round 后的标签和颜色；有对应方法时，右侧同步出现。最后出现 Clean Up Context Prompt。不需要逐字朗读每个气泡。

我们先看左边整段对话。最开始只是让 AI 用 source files review AE summary，后来补充说先检查 counts 和 percentages，不要把重点放在 formatting。接下来又提到 appendix、旧版 AE table、当前文件，中间还插了一句给团队写邮件。到了最后，我们只是说，回到 AE QC brief 再改一下。

站在我们的角度，好像一直在推进同一个任务。但对于 AI 来说，哪些要求仍然有效，哪些是之前的版本，哪些只是另外一件事，就需要被明确区分。

我们再一步一步看这些标签。前两轮是原始任务和补充要求，都是理解当前目标的重要信息。到了 Round 6，关于 formatting 的补充可以整理成一条当前规则，比如主体先讲数据问题，formatting 放 appendix，不需要每次都重新解释整段讨论。这对应右边的 Compress。

Round 9 提到了旧表，Round 12 又说只用当前文件。这时候要确认最新的 source of truth 是什么。已经被替代的版本，可以从当前工作依据中移出去，这对应 Remove；如果版本或要求之间仍有冲突，就要 Resolve，不能让 AI 自己随便挑一个。

再看 Round 16，给团队写邮件是另一项任务，不应该一直混在 AE QC 的主任务里。这里的 Remove 是不再把它作为当前 review 的有效要求，不是让 AI 删除原文件或聊天记录。

Round 19 提到 earlier tool output。我们需要保留的是已经核实的 counts、计算依据和来源，较长的原始输出可以压缩成可检查的结论；真的需要细节时，再回到对应文件。这也就是 Compress 和 Retrieve 的配合。到最后一个当前请求，我们要让 AI 根据当前有效规则继续，按需要读取支持材料。

那了解这些方法以后，我们可以实际对 AI 说什么？右下这个浅蓝色的 Clean Up Context Prompt，就是可以拿走用的。

它要求 AI 先检查当前能看到的 conversation 和 files，整理一份简短的 Working Context：现在的 goal、source-of-truth files、confirmed rules、verified findings、open issues、哪些信息已经失效，以及 next step。这里我们还特别要求保留 source references，不要编造决定，不要擅自解决有歧义的冲突。如果仍然判断不了，再问我们。

放回这个 AE case，整理出来以后应该能够看清楚：当前 review 哪份 RTF，统计规则来自哪个 spec，Placebo safety N=5 这个 finding 依据什么，以及哪些代码问题还没有验证。我们先看这份 brief 是否准确，再继续 analysis，避免它一边整理、一边又沿着旧理解往下做。

所以这个 prompt 适合在当前 conversation 开始跑偏时，用来重新明确有效信息。下一页则更进一步：如果想换一个新的 conversation，怎样把必要的状态交接过去?

Page 8 | The Context Reset Protocol

> **讲图提示：**旧对话卡片 → STOP 四步 → Create a Handoff Prompt → 白色案例简报 → 同位置切换为 Resume from a Handoff Prompt。两个蓝色 prompt 是先后使用的两次指令。

左上这些叠起来的卡片，表示一个可能已经聊了很多轮、改过几次要求、引用过多个版本的 conversation。这里的 21 rounds、3 file versions 是讲解整理前状态的示意，不是说我们这个 mock case 真正发生过这么多轮。

如果已经很 messy，我们可以先用左边这个 STOP method。Stop，先不要继续加要求；Take Stock，确认目标、进度和问题；Organize，把有效信息、过时内容和未决问题分开；最后 Proceed Cleanly，带着清楚的当前状态继续工作。

那具体怎么让 AI 帮我们做？先看右上的 Create a Handoff Prompt。它要求 AI 为一个 fresh conversation 整理六个部分：Goal、Current State、Decisions、Open Issues、Ignore 和 Next Step。还要说明当前依据哪些文件，保留已经验证的数值和来源，不要把 open issues 自动补成 decisions。此时只返回 brief，不继续做任务。

下面白色这块，就是用 AE summary 展开的示例。Goal 是完成这份 AE QC brief。Current State 里面写清楚五个文件，以及已经核实的 Placebo safety N=5，但 table 写的是 N=6。Decisions 保留 SAFFL=Y、subject-level counting、分母和排序这些已有规则。

Open Issues 则明确：具体代码缺陷还需要 programmer 检查，corrected output 和 rerun log 还没有拿到，release approval 也没有完成。Ignore 里写清楚旧表、失效规则和单独的邮件任务，不要把它们带回当前分析。最后 Next Step 才是继续修订 QC brief，保留证据，代码修改和 release decision 仍然等待确认。

我觉得 Open Issues 和 Ignore 很重要。我们交接的不只是要记住什么，也包括什么还没有决定、什么不再有效。否则新对话很容易把一句建议理解成已经确认的结论。

接下来，右上换成第二个 prompt：Resume from a Handoff。这一段是到了新 conversation 以后使用的。我们把刚才检查过的 brief，以及实际需要的文件或可访问的引用交给它，让它先读，再确认当前 goal、accepted decisions、open issues 和 next step。

这里还要提醒它：文件缺失或者内容冲突，要先指出；不要把未决问题当成决定，也不要在没有验证需要的情况下重复已经完成的工作。先把理解和准备做的下一步说出来，等我们确认后再执行。只贴一份 brief，并不意味着新对话已经自动拿到了旧对话里的附件，所以文件是否实际可访问也要检查。

这两个 prompt 的分工就是：第一个帮我们生成交接内容，第二个帮接手的 AI 根据这份内容恢复任务。第七页偏向在当前对话里清理上下文，这页偏向把状态带到一个新的起点。

如果是周期比较长的项目，也可以从一开始维护类似 memory.md 的工作记录，持续更新进度、决定、open issues 和 next step。核心是把历史 conversation 整理成当前任务需要继续的 state，而不是简单把所有聊天再复制一遍。

Page 9 | Output Design — Same Finding, Different Audiences

> **讲图提示：** 原始 AI 输出 → 信息层级的说明 → 三张白色受众示例卡片 → 蓝色 Output Contract Prompt → 同位置切换为 Output Repair Prompt。

前面解决了任务和上下文的问题，但即使 AI 用对了信息，最后还有一个很实际的问题：这个 Output 到底好不好用?

上面这段 AI 输出，把 Placebo denominator、Any AE、GI 和 Nausea 的 counts，以及 PT sorting 放在一起。里面的信息可能都有价值，但 programmer、statistician 和 Leader，不一定需要按照同一个顺序看它们。

我们用同一个 finding 对照下面三张白色卡片：Placebo Any AE，table 是 5 (83.3%)，独立 QC 算出来是 4 (80.0%)。这三张是面向不同受众的示例输出，不是可以直接复制给 AI 的 prompt，也不表示这些交付已经真实发送或审批过。

Programmer 先需要知道 discrepancy 在哪里。哪张表、哪个 treatment group、哪一行，现在是多少，QC 应该是多少？后面给出依据：ADSL 的 safety N=5，ADAE 里符合规则、有 AE 的 distinct subjects 是四个。然后说明 next step：检查 population 和 counting logic，重新生成 table，再核对 output 和 log。

Statistician 更先关注这个比较使用了什么定义。是否按照 spec 使用 safety population，Any AE 是否按 subject 去重？在这个定义下，4/5 对应 80.0%。后面还要核对 corrected table 是否符合 spec，不能从这组很小的 mock data 直接推导 treatment effect 或因果结论。

Leader 则更需要知道这件事对 deliverable 有什么影响。现在存在已经确认的差异，programming correction、rerun 和 QC reconciliation 还没有完成。接下来需要确认由谁处理、什么时候 review，再决定是否批准交付。这里是把需要的 action 和 decision 放在前面，没有自己编一个 owner 或日期。

所以 Output Design 的重点不是简单地缩短，而是同一份 evidence，根据使用者重新组织 information hierarchy。

下面先出现的是 Output Contract Prompt，用在生成之前。里面的方括号是要替换的内容：Audience 是谁，Purpose 或 Decision 是什么，优先哪些信息，用什么 Format，控制在什么 Length。同时保留 verified values 和 source references，把 findings、interpretation 和 unresolved questions 分开。

放回这个 case，如果给 programmer，我们就把 Audience 换成 statistical programmer，优先级设成 location、discrepancy、evidence、action，格式用 issue list。如果给 Leader，就先讲 deliverable impact、pending work 和 decision needed。我们先定义这个 contract，AI 才知道什么应该放前面。

再点击，换成 Output Repair Prompt。这个是在已经有了一份 verified analysis、但表达方式不合适的时候用的。它要求保留已经核实的数值，重排内容，把重要 findings 放前面，细节放 appendix，未知项单独标明，不重新做 analysis，也不自行添加原因。

这两段分别解决生成前和生成后的问题：先把输出要求定义好；如果现有内容不好用，就针对表达方式修订。但不管整理得多清楚，最后还要问，它的结论有没有依据?

Page 10 | Evidence Check — Fact, Inference, Unknown

> **讲图提示：** AI 原话 → 证据对照和解释 → 三类结论 → 核查表 → Evidence Check Prompt。第一句红色文字是待检查的过度结论，不要作为事实朗读。

我们看上面 AI 这句话。它说 Placebo percentage 不对，接着就说“已经确认具体的 SAS code defect”。前半句和后半句，需要的 evidence 其实不一样。

这里继续用同一组 AE 数据。Table 里面 Placebo Headache 是 2/6，也就是 33.3%。按照 ADSL 里的 SAFFL=Y，Placebo safety N 应该是五；ADAE 里有 Headache 的 safety subjects 是两个人，所以重算应该是 2/5，也就是 40.0%。因此我们可以直接确认：这里的 denominator 和 percentage 不一致。

但这不等于已经确认了具体源码缺陷。这个 case 的 log 确实提示 Placebo denominator 是在 SAFFL filter 之前生成的，这条 warning 是存在的，也给了我们检查方向。但我们没有在这些文件里看到 SAS source code，也还没有拿到修正后的 rerun 结果，所以不能进一步说具体哪行代码有错、其他 count 的问题也都是同一个原因，或者修复已经成功。

这时候就可以把 key claims 分成三类。Fact 是有直接 evidence 的内容，比如 table 用 N=6，而 ADSL safety population 是 N=5。Inference 是合理但仍需验证的解释，比如 logged warning 可能解释一部分 percentage errors。Unknown 则是当前证据还不能确定的具体 code defect 和修正效果。

下面这个小表，实际是在把 statement 和 evidence 的关系摊开。你说 denominator inconsistent，可以回到 table 和 dataset 核对；你说 exact code defect confirmed，就要问，source code 在哪里，缺的验证是什么?

最后这个浅蓝色的 Evidence Check Prompt，就可以放在准备交付之前，让 AI 对每个 key claim 标出 FACT、INFERENCE 或 UNKNOWN。Fact 要给出 source，Inference 要说明还需要验证什么，证据不足就保留为 Unknown，不要补成一个 conclusion。

当然，AI 自己标了 Fact，不等于这句话就已经通过 QC。这个 prompt 的作用，是把它的 claim 和 evidence 显示出来，方便我们回到 source 去检查，找到它是从哪一步开始做了没有依据的推断。

Page 11 | Conclusion

最后把今天这部分连起来看：Structured Prompt 帮我们说明该做什么，Managed Context 帮我们整理当前应该依据哪些信息，Verifiable Output 则让我们能够回到 source，检查结论从哪里来。

放在这个 AE case 里面，就是先明确文件角色和统计规则，接着在反复讨论时维护当前有效的要求，最后把 findings 整理成不同读者可以使用、也可以核查的结果。

今天这些浅蓝色 prompt，不需要每次全部用一遍。任务还不清楚，就先整理需求；对话开始跑偏，就清理 context；要换 conversation，就先做 handoff 再恢复；要生成结果，就先定义 output contract；已有结果不好用，再做 output repair；交付前，最后检查 evidence。

我们希望留下的，是这几种情况下可以采取的具体动作。从和 AI 来回聊天，开始走向一个能够继续、能够检查，也能逐步复用的工作方式。